JOINS

INNER JOIN

SQL

- Esistono diversi tipi di JOIN, esamineremo il tipo di JOIN più semplice, che è un INNER JOIN

SQL

- Cos'è un'operazione JOIN?
- I JOIN ci consentono di combinare più tabelle insieme.

- La nostra azienda sta tenendo una conferenza per le persone nel settore del noleggio di film.
- Faremo in modo che le persone si registrino online in anticipo e quindi accedano il giorno della conferenza.

SQL

- Dopo la conferenza abbiamo le tabelle

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- id indicano in quale ordine si sono registrati o hanno effettuato l'accesso al sito

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- Per semplicità, consideriamo i nomi univoci

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- INNER JOIN ritornerà un set di record che contiene dati comuni a entrambe le tabelle

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

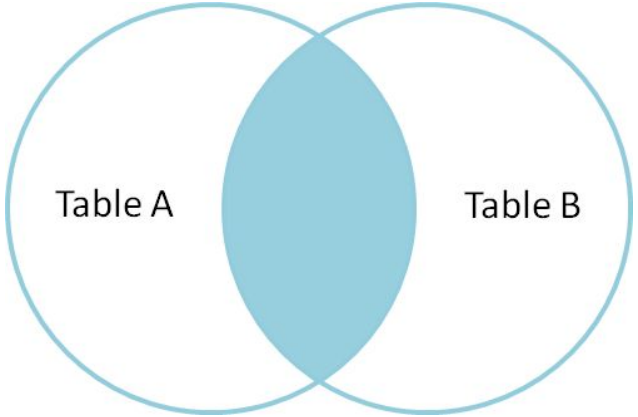
LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- SELECT * FROM **TableA**
  INNER JOIN **TableB**
  ON **TableA**.col_match = **TableB**.col_match

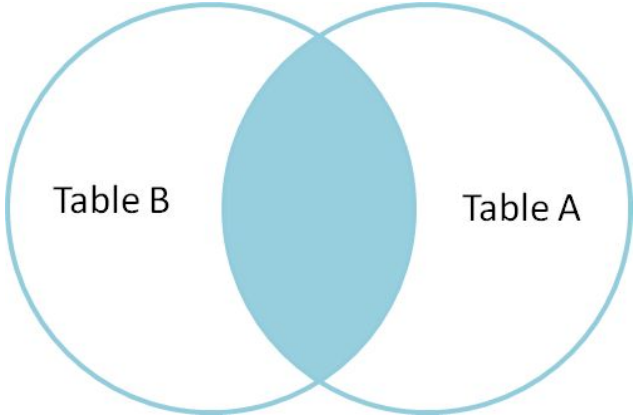
SQL

- SELECT * FROM **TableA**
  INNER JOIN **TableB**
  ON **TableA**.col_match = **TableB**.col_match



SQL

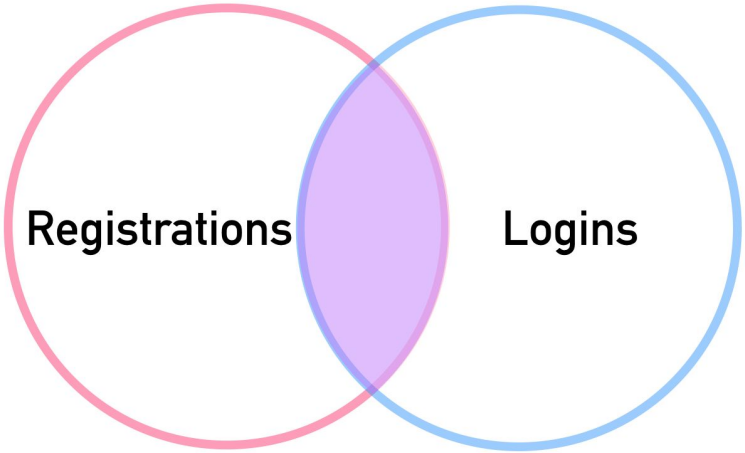
- SELECT * FROM **TableB**
  INNER JOIN **TableA**
  ON **TableA**.col_match = **TableB**.col_match



SQL

- SELECT * FROM Registrations
  INNER JOIN Logins
  ON Registrations.name = Logins.name

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

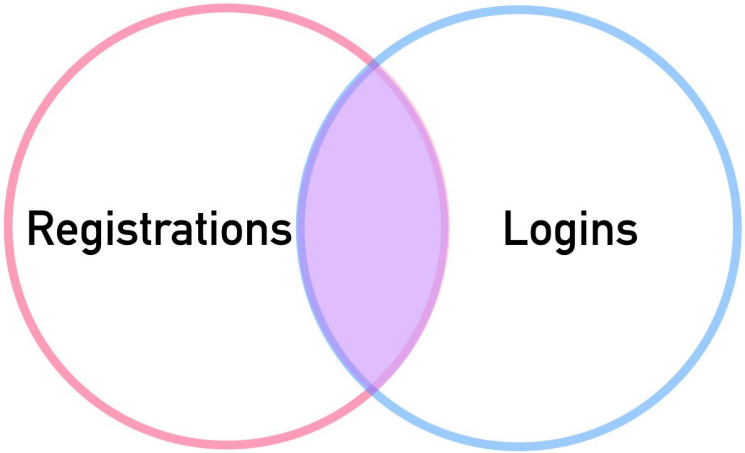


LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- SELECT * FROM Registrations
  INNER JOIN Logins
  ON Registrations.name = Logins.name

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

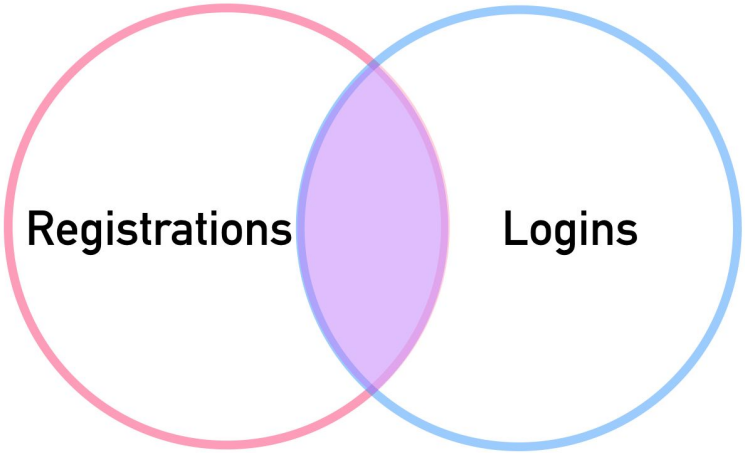


LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- SELECT * FROM Registrations
  INNER JOIN Logins
  ON Registrations.name = Logins.name

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David



LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- SELECT * FROM Registrations
  INNER JOIN Logins
  ON Registrations.name = Logins.name

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

RESULTS			
reg_id	name	log_id	name
1	Andrew	2	Andrew
2	Bob	4	Bob

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- SELECT reg_id,Logins.name,log_id
  FROM Registrations
  INNER JOIN Logins
  ON Registrations.name = Logins.name

RESULTS		
reg_id	name	log_id
1	Andrew	2
2	Bob	4

SQL

- SELECT reg_id,Logins.name, og_id
  FROM Registrations
  INNER JOIN Logins
  ON Registrations.name = Logins.name

RESULTS		
reg_id	name	log_id
1	Andrew	2
2	Bob	4

SQL

- l'ordine della tabella non ha importanza in un INNER JOIN

OUTER JOINS

SQL

- Esistono tipi diversi di OUTER JOIN
- Ci consente di specificare come trattare i valori presenti solo in una delle tabelle che vengono unite.

FULL OUTER JOIN

SQL

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- Andrew e Bob sono presenti in entrambe

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- Ma ci sono anche nomi presenti in una sola tabella

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- Vediamo come si comportano i diversi tipi di OUTER JOIN

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

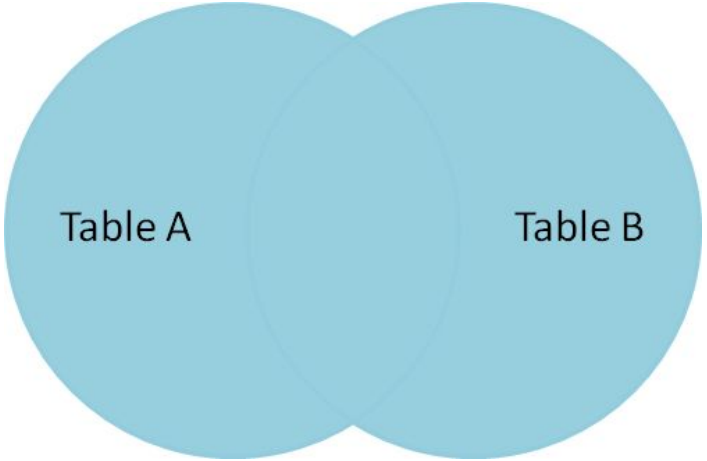
FULL OUTER JOIN

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

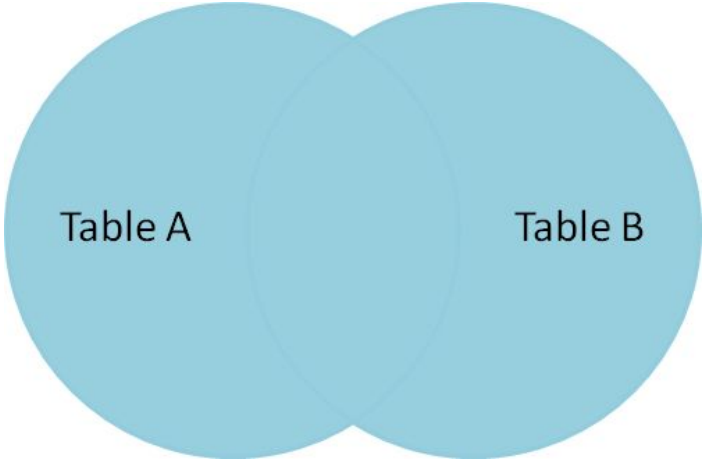
SQL

- SELECT * FROM **TableA**
  FULL OUTER JOIN **TableB**
  ON **TableA**.col_match = **TableB**.col_match



SQL

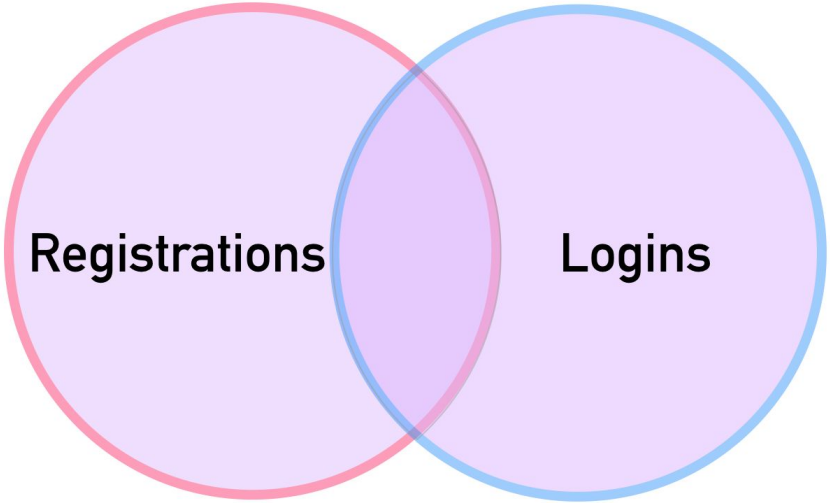
- SELECT * FROM **TableB**
  FULL OUTER JOIN **TableA**
  ON **TableA**.col_match = **TableB**.col_match



SQL

- SELECT * FROM Registrations
  FULL OUTER JOIN Logins
  ON Registrations.name = Logins.name

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David



LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- SELECT * FROM Registrations FULL OUTER JOIN Logins ON Registrations.name = Logins.name

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- SELECT * FROM Registrations FULL OUTER JOIN Logins ON Registrations.name = Logins.name

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

RESULTS			
reg_id	name	log_id	name
1	Andrew		
2	Bob		
3	Charlie		
4	David		

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- SELECT * FROM Registrations FULL OUTER JOIN Logins ON Registrations.name = Logins.name

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

RESULTS			
reg_id	name	log_id	name
1	Andrew	2	Andrew
2	Bob	4	Bob
3	Charlie		
4	David		

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- SELECT * FROM Registrations FULL OUTER JOIN Logins ON Registrations.name = Logins.name

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

RESULTS			
reg_id	name	log_id	name
1	Andrew	2	Andrew
2	Bob	4	Bob
3	Charlie	null	null
4	David	null	null

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- SELECT * FROM Registrations FULL OUTER JOIN Logins ON Registrations.name = Logins.name

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

RESULTS			
reg_id	name	log_id	name
1	Andrew	2	Andrew
2	Bob	4	Bob
3	Charlie	null	null
4	David	null	null
null	null	1	Xavier
null	null	3	Yolanda

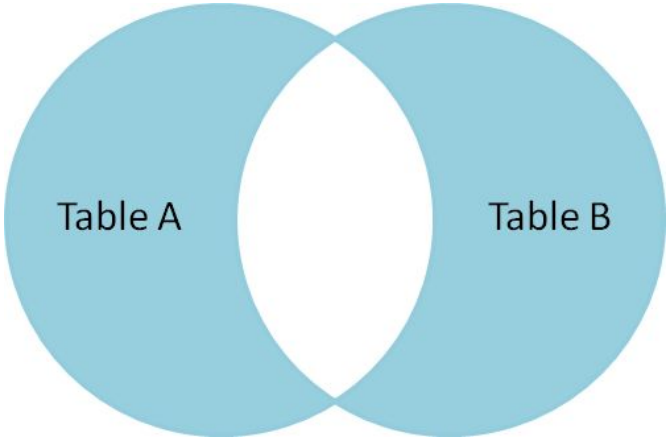
LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

FULL OUTER JOIN con WHERE

Ottieni righe univoche per entrambe le tabelle
(righe non trovate in entrambe le tabelle)

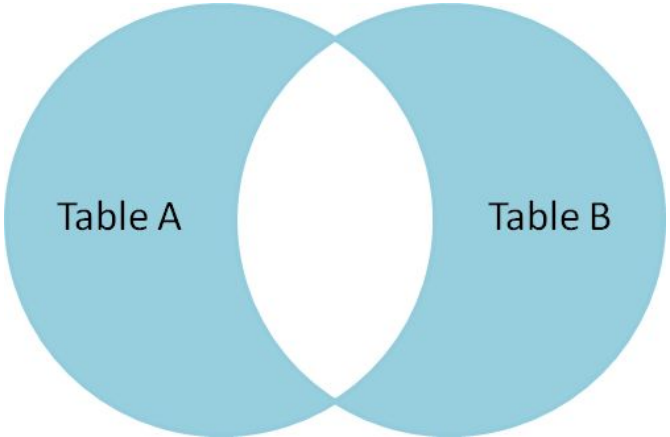
SQL

- SELECT * FROM **TableA**
  FULL OUTER JOIN **TableB**
  ON **TableA**.col_match = **TableB**.col_match
  WHERE **TableA**.id IS null OR
  TableB.id IS null



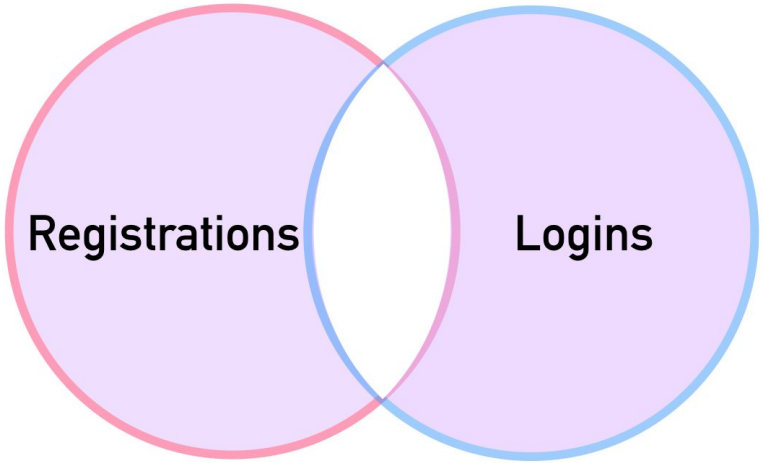
SQL

- SELECT * FROM **TableB**
  FULL OUTER JOIN **TableA**
  ON **TableA**.col_match = **TableB**.col_match
  WHERE **TableA**.id IS null OR
  TableB.id IS null



SQL

```
SELECT * FROM Registrations FULL OUTER
JOIN Logins
ON Registrations.name = Logins.name
WHERE Registrations.reg_id IS null OR
Logins.log_id IS null
```



SQL

- SELECT * FROM Registrations FULL OUTER JOIN Logins ON Registrations.name = Logins.name

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

RESULTS			
reg_id	name	log_id	name
1	Andrew	2	Andrew
2	Bob	4	Bob
3	Charlie	null	null
4	David	null	null
null	null	1	Xavier
null	null	3	Yolanda

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- SELECT * FROM Registrations FULL OUTER JOIN Logins ON Registrations.name = Logins.name WHERE Registrations.reg_id IS null OR Logins.log_id IS null

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

RESULTS			
reg_id	name	log_id	name
3	Charlie	null	null
4	David	null	null
null	null	1	Xavier
null	null	3	Yolanda

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

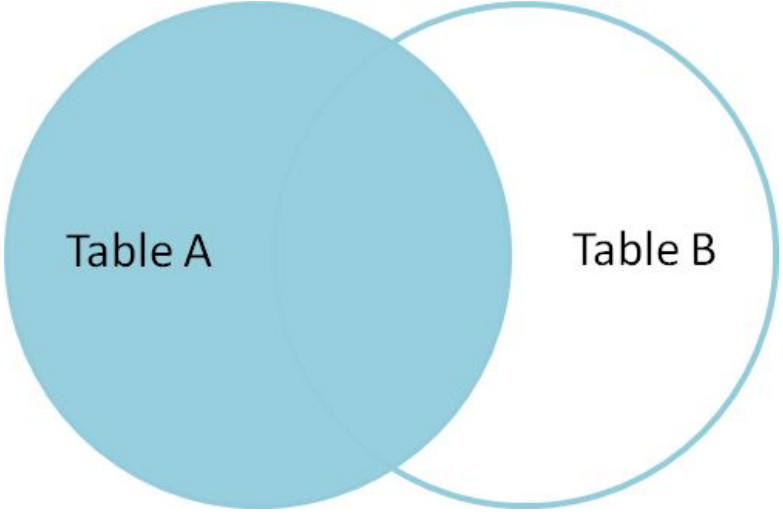
LEFT OUTER JOIN

SQL

- LEFT OUTER JOIN ritorna un set di record che si trovano nella tabella di sinistra, se non c'è corrispondenza con la tabella di destra, i risultati sono nulli.
- Più avanti impareremo come aggiungere istruzioni WHERE per modificare ulteriormente un LEFT OUTER JOIN

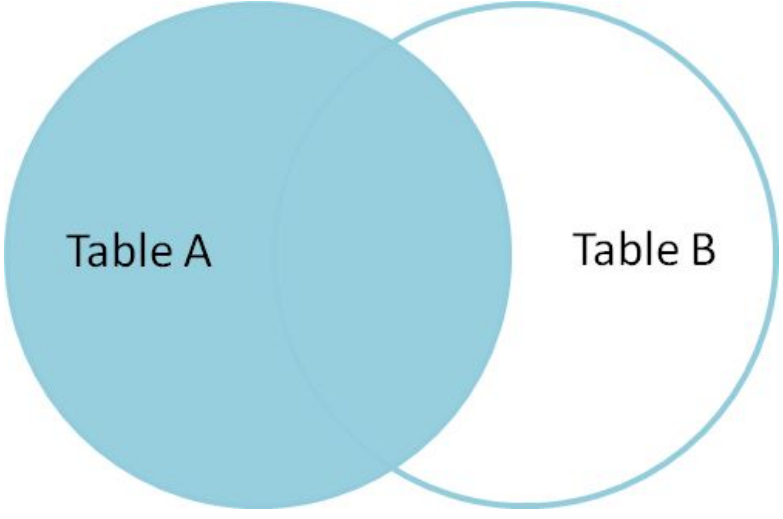
SQL

- SELECT * FROM **TableA**
  LEFT OUTER JOIN **TableB**
  ON **TableA**.col_match = **TableB**.col_match



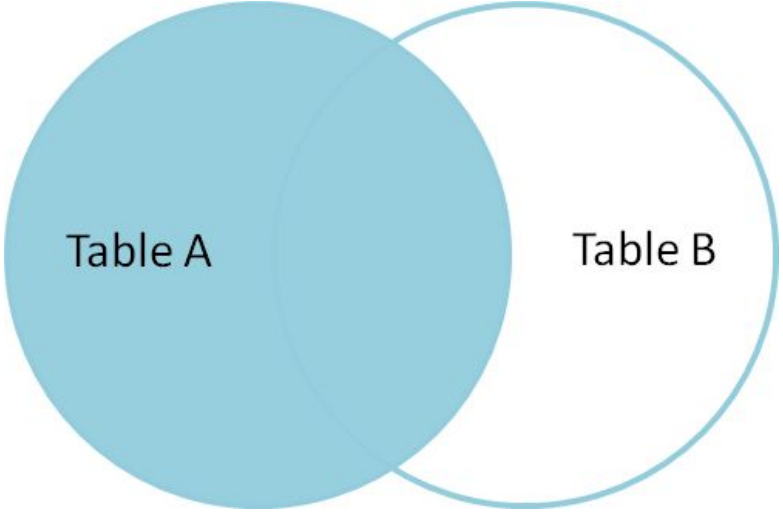
SQL

- SELECT * FROM **TableA**
  LEFT JOIN **TableB**
  ON **TableA**.col_match = **TableB**.col_match



SQL

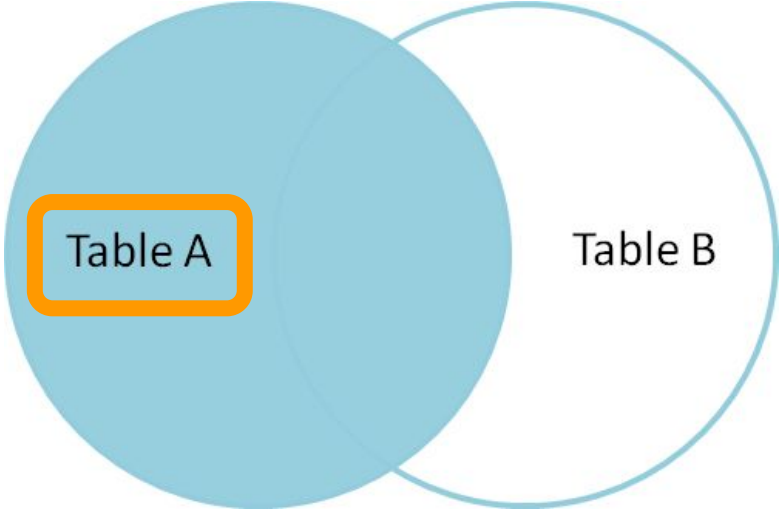
- SELECT * FROM **TableA**
  LEFT OUTER JOIN **TableB**
  ON **TableA**.col_match = **TableB**.col_match



SQL

L'ordine conta per LEFT OUTER JOIN!

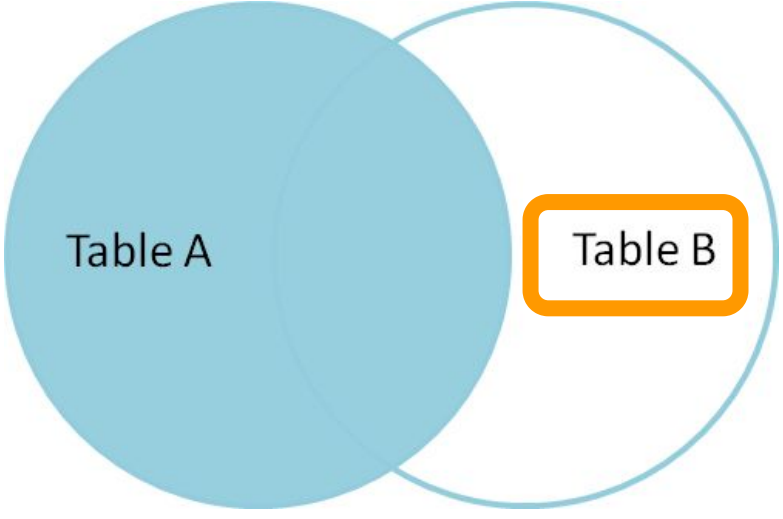
- SELECT * FROM **TableA**
  LEFT OUTER JOIN **TableB**
  ON **TableA**.col_match = **TableB**.col_match



SQL

L'ordine conta per LEFT OUTER JOIN!

- SELECT * FROM **TableA**
  LEFT OUTER JOIN **TableB**
  ON **TableA**.col_match = **TableB**.col_match



SQL

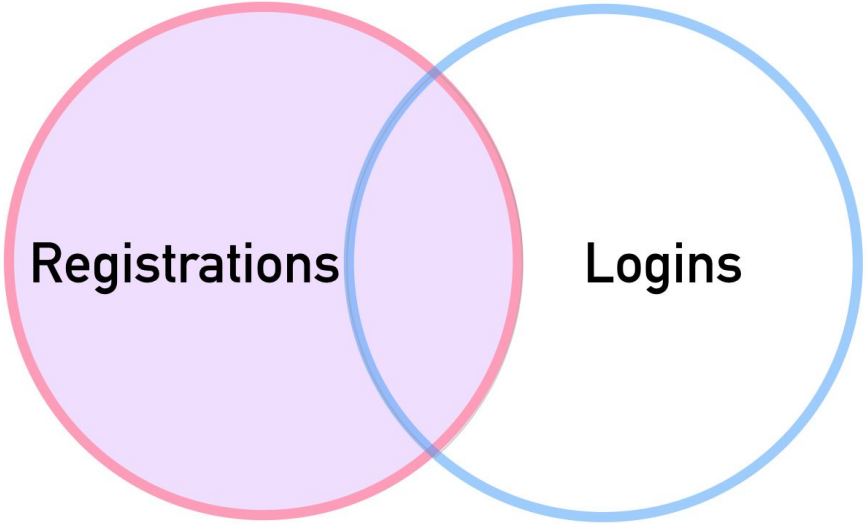
REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- SELECT * FROM Registrations
  LEFT OUTER JOIN Logins
  ON Registrations.name = Logins.name

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David



LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- SELECT * FROM Registrations
  LEFT OUTER JOIN Logins
  ON Registrations.name = Logins.name

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

RESULTS			
reg_id	name	log_id	name
1	Andrew	2	Andrew
2	Bob	4	Bob
3	Charlie	null	null
4	David	null	null

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

- SELECT * FROM Registrations
  LEFT OUTER JOIN Logins
  ON Registrations.name = Logins.name

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

RESULTS			
reg_id	name	log_id	name
1	Andrew	2	Andrew
2	Bob	4	Bob
3	Charlie	null	null
4	David	null	null

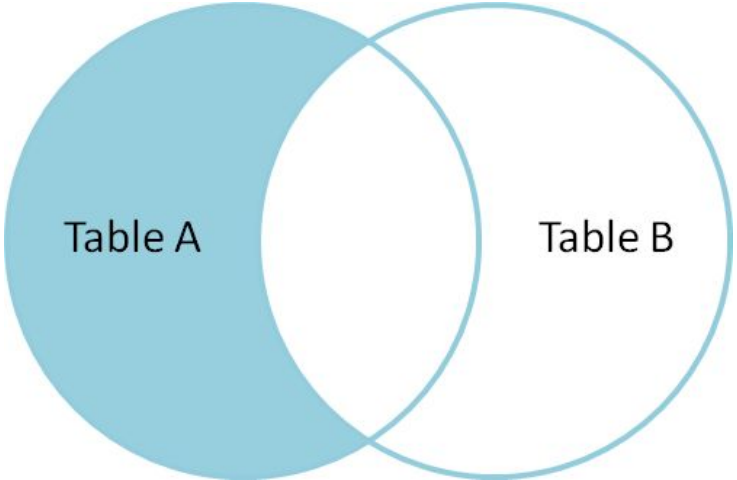
LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

LEFT OUTER JOIN con WHERE

Righe univoche per la tabella sinistra

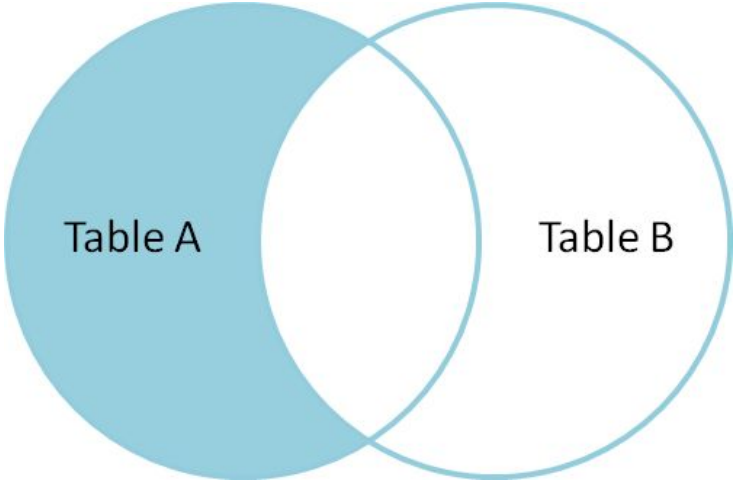
SQL

- E se volessimo solo voci univoche per la tabella A? Quelle righe trovate nella tabella A e non trovate nella tabella B.



SQL

- SELECT * FROM **TableA**
  LEFT OUTER JOIN **TableB**
  ON **TableA**.col_match = **TableB**.col_match
  WHERE **TableB**.id IS null



SQL

- SELECT * FROM Registrations
  LEFT OUTER JOIN Logins
  ON Registrations.name = Logins.name

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

RESULTS			
reg_id	name	log_id	name
1	Andrew	2	Andrew
2	Bob	4	Bob
3	Charlie	null	null
4	David	null	null

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

SQL

```
SELECT * FROM Registrations
LEFT OUTER JOIN Logins
ON Registrations.name = Logins.name
WHERE Logins.log_id IS null
```

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

RESULTS			
reg_id	name	log_id	name
3	Charlie	null	null
4	David	null	null

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

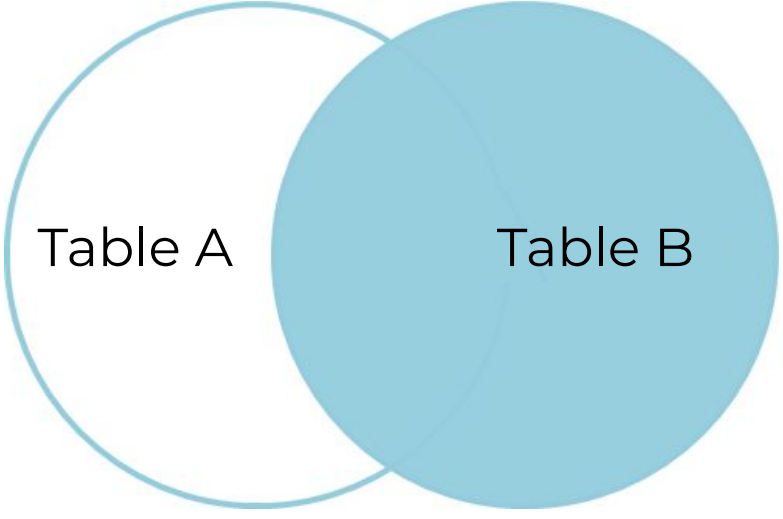
RIGHT JOINS

SQL

- Un RIGHT JOIN è essenzialmente lo stesso di un LEFT JOIN, tranne per il fatto che le tabelle vengono scambiate.

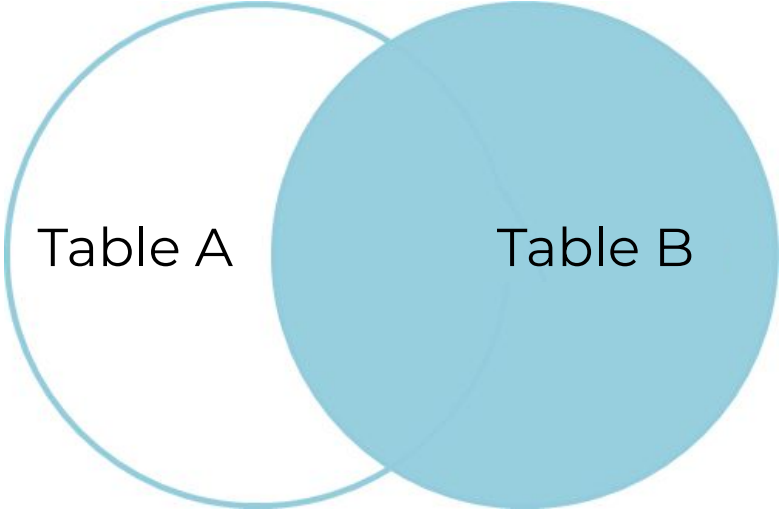
SQL

- SELECT * FROM **TableA**
  RIGHT OUTER JOIN **TableB**
  ON **TableA**.col_match = **TableB**.col_match



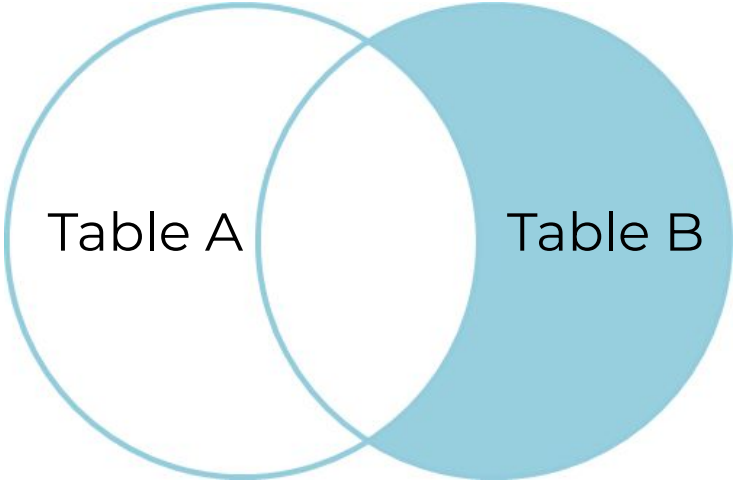
SQL

- SELECT * FROM **TableA**
  RIGHT JOIN **TableB**
  ON **TableA**.col_match = **TableB**.col_match



SQL

- SELECT * FROM **TableA**
  RIGHT OUTER JOIN **TableB**
  ON **TableA**.col_match = **TableB**.col_match
  WHERE **TableA**.id IS null



JOIN

CHALLENGE TASKS

UNIONS

SQL

- L'operatore UNION viene utilizzato per combinare il set di risultati di due o più istruzioni SELECT.

SQL

- SELECT column_name(s) FROM table1
  UNION
  SELECT column_name(s) FROM table2;

SQL

Sales2021_Q1	
name	amount
David	100
Claire	50

Sales2021_Q2	
name	amount
David	200
Claire	100

SQL

- SELECT * FROM Sales2021_Q1
  UNION
  SELECT * FROM Sales2021_Q2;

name	amount
David	100
Claire	50
David	200
Claire	100

SQL

- SELECT * FROM Sales2021_Q1
  UNION
  SELECT * FROM Sales2021_Q2
  ORDER BY name;

name	amount
David	100
David	200
Claire	50
Claire	100

JOIN Challenges

Quesito 1

- Le leggi sull'imposta sulle vendite della California sono cambiate e dobbiamo avvisare i nostri clienti tramite e-mail.
- Quali sono le email dei clienti che vivono in California?

SQL

- Risultato atteso

	district character varying (20)	email character varying (50)
1	California	patricia.johnson@sakilacust...
2	California	betty.white@sakilacustomer...
3	California	alice.stewart@sakilacustom...
4	California	rosa.reynolds@sakilacusto...
5	California	renee.lane@sakilacustomer....
6	California	kristin.johnston@sakilacust...
7	California	cassandra.walters@sakilacu...
8	California	jacob.lance@sakilacustome...
9	California	rene.mcalister@sakilacusto...

SQL

- Suggerimenti
  - Sarà necessario utilizzare le tabelle degli indirizzi e dei clienti.
  - Guarda la colonna *district*

SQL

```
SELECT district,email FROM address
INNER JOIN customer ON
address.address_id = customer.address_id
WHERE district = 'California'
```

Quesito 2

- Un cliente entra ed è un grande fan dell'attore "Nick Wahlberg" e vuole sapere in quali film recita.
- Ottieni un elenco di tutti i film in cui è stato "Nick Wahlberg".

SQL

- Risultato atteso

	title character varying (255)	first_name character varying (45)	last_name character varying (45)
1	Adaptation Holes	Nick	Wahlberg
2	Apache Divine	Nick	Wahlberg
3	Baby Hall	Nick	Wahlberg
4	Bull Shawshank	Nick	Wahlberg
5	Chainsaw Uptown	Nick	Wahlberg
.....			
21	Mask Peach	Nick	Wahlberg
22	Roof Champion	Nick	Wahlberg
23	Rushmore Mermaid	Nick	Wahlberg
24	Smile Earring	Nick	Wahlberg
25	Wardrobe Phantom	Nick	Wahlberg

SQL

- SUGGERIMENTI
  - Dovrai eseguire 2 JOIN di fila per eseguire questa operazione in una singola query.
  - Tabelle da utilizzare: attore, film, film_attore

SQL

```
SELECT title,first_name,last_name
FROM film_actor INNER JOIN actor
ON film_actor.actor_id = actor.actor_id
INNER JOIN film
ON film_actor.film_id = film.film_id
WHERE first_name = 'Nick'
AND last_name = 'Wahlberg'
```